

# Capitolo 14 — Come WCM individua i protocolli da applicare

|                    |                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book                                           |
| <b>Versione</b>    | FROZEN-14                                                                                             |
| <b>Data master</b> | 2026-08-29                                                                                            |
| <b>Stato</b>       | FROZEN                                                                                                |
| <b>Source path</b> | wcm/documentation/process-memory-book/<br>chapters/14_come_wcm_individua_i_protocolli_da_applicare.md |
| <b>Source SHA</b>  | f4c0b0e                                                                                               |
| <b>Release</b>     | 2026-09-04T09:44:14.164Z                                                                              |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

---

**Stato:** FROZEN **Parte:** V — Da una richiesta alle regole applicabili **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

---

## 14.0 Il problema non è conoscere i protocolli. È sapere quando servono

Nel Capitolo 13 abbiamo seguito una richiesta dall'intenzione fino all'esecuzione.

A un certo punto della route compare una domanda inevitabile:

### **| Quali protocolli devo applicare adesso?**

Il WCM corrente possiede un Protocol Book.

Ma il fatto che un protocollo esista non significa che debba essere caricato in ogni attività.

Se ogni agente, a ogni richiesta, leggesse tutti i protocolli disponibili, avremmo ricreato lo stesso problema che INDEX-FIRST risolve per la conoscenza generale:

- troppo contesto;
- molte regole irrilevanti;
- maggiore costo cognitivo;
- più possibilità di confondere scope differenti;
- difficoltà nel distinguere una regola realmente applicabile da una semplicemente esistente.

La risposta WCM non è quindi:

**|** «Conosci tutti i protocolli.»

È:

**|** «Individua i protocolli applicabili alla situazione corrente attraverso il routing minimo e autorevole.»

Questo capitolo spiega come.

---

## 14.1 Protocol routing

Un protocollo WCM è una regola operativa con un proprio scope.

Può definire:

- una guard;
- un obbligo;
- una verifica;
- una stop condition;
- una policy di delega;
- una regola di retrieval;
- una condizione di authority;
- una disciplina di mutazione persistente;
- una modalità di gestione di failure o dipendenze.

Il **Protocol Routing** è il meccanismo con cui WCM collega una situazione operativa al sottoinsieme di protocolli che devono governarla.

La formula di base è:

```
SITUAZIONE
↓
TIPO DI OPERAZIONE / EVENTO
↓
HOOK APPLICABILE
↓
PROCESSI + PROTOCOLLI PERTINENTI
↓
AZIONE / GUARD / STOP
```

Nel WCM esistono due livelli complementari:

```
ROUTING COGNITIVO
= comprendere che tipo di situazione stiamo affrontando

ROUTING DETERMINISTICO
= quando l'evento è già strutturato,
  caricare esattamente la route dichiarata
```

Non sono concorrenti.

Il primo dà significato.

Il secondo evita di reinterpretare ogni volta ciò che il sistema ha già formalizzato.

---

## 14.2 Regole trasversali

Alcuni protocolli sono legati a una fase molto specifica.

Altri attraversano molti processi differenti.

Per esempio, una regola come INDEX-FIRST può diventare pertinente in molte attività, perché riguarda **come recuperare il contesto**.

Una regola di Persistent Mutation Safety può diventare pertinente ogni volta che si sta per modificare stato persistente rilevante.

Una regola di Contiguous Workflow Execution può diventare pertinente quando esiste un workflow già avviato.

Questi protocolli possono essere definiti **trasversali** perché non appartengono a un solo processo.

Ma trasversale non significa:

| «sempre caricato».

Significa:

| **può applicarsi in molti contesti diversi quando il suo trigger è presente.**

Questa distinzione è essenziale.

TRASVERSALE  
≠  
UNIVERSALE IN OGNI RUN

Un protocollo può avere ampia applicabilità e, nello stesso tempo, essere irrilevante per il task corrente.

## 14.3 Protocolli condizionali

Molti protocolli diventano necessari soltanto quando si verifica una determinata condizione.

Esempio astratto:

OPERAZIONE NORMALE  
→ nessun tool failure  
→ nessuna route di capability failure necessaria

Poi avviene qualcosa:

TOOL FAILURE  
→ capability non ancora verificata  
→ protocollo di capability evidence applicabile

La condizione modifica il routing.

Non perché il protocollo sia diventato "più importante" in assoluto.

Perché è diventato **pertinente a un evento che prima non esisteva.**

Questo rende il routing dinamico.

Un task può iniziare con tre regole applicabili e incontrarne altre durante il proprio sviluppo.

Il contesto procedurale può quindi crescere progressivamente:

START  
→ protocol set minimo  
  
EVENTO NUOVO  
→ route aggiuntiva  
  
NUOVO EVENTO  
→ eventuale nuova route  
  
STOP

→ nessun caricamento ulteriore

È lo stesso principio del Progressive Retrieval applicato alla procedura.

---

## 14.4 Trigger espliciti

Il caso più semplice è quando il trigger è dichiarato in modo esplicito.

Una richiesta potrebbe dire:

| «*Verifica la capability prima di dichiarare il blocco.*»

Oppure:

| «*Porta il workflow al Board Gate.*»

Oppure ancora:

| «*Aggiorna la documentazione dopo il delta materiale.*»

In questi casi il linguaggio della richiesta o il workflow stesso può rendere evidente quale famiglia di regole è coinvolta.

Ma il trigger esplicito non autorizza a saltare Source Precedence e status check.

Se la richiesta nomina un protocollo superseded, la baseline corrente deve prevalere.

Se la richiesta nomina un comportamento incompatibile con governance, il nome del protocollo non crea authority.

Il trigger aiuta il routing.

Non sostituisce il controllo della fonte.

---

## 14.5 Trigger derivati dal tipo di operazione

Molto spesso l'utente non nomina alcun protocollo.

Dice semplicemente:

| «*Modifica questo file.*»

Oppure:

| «*Continua il workflow.*»

Oppure:

| «*Non riesco a recuperare il contenuto.*»

WCM deve derivare il routing dal **tipo di operazione**.

Esempio:

OPERAZIONE  
persistent mutation

DERIVAZIONE  
questa azione modifica stato persistente

ROUTE  
carica le guard di persistent mutation applicabili

Oppure:

```
OPERAZIONE
resume di workflow incompleto

DERIVAZIONE
esiste next_transition persistita

ROUTE
bootstrap + retrieval + contiguous execution
```

Il passaggio cognitivo è:

**| riconoscere la classe dell'operazione.**

Una volta riconosciuta, la parte successiva può essere molto più meccanica.

## 14.6 Process → Protocol relationships

Processi e protocolli non sono due cataloghi separati che vivono senza relazioni.

Un processo può dipendere da più protocolli.

Un protocollo può governare più processi.

La relazione è quindi multi-a-multi.

```
PROCESSO A
├─ PROTOCOLLO 1
├─ PROTOCOLLO 2
└─ PROTOCOLLO 3

PROCESSO B
├─ PROTOCOLLO 2
└─ PROTOCOLLO 4
```

Il Process Register WCM espone infatti relazioni operative tra baseline di processi e protocolli.

Per esempio, la navigazione Agent-Ready viene collegata a:

```
CONCEPT-007
+ PROC-005
+ PROT-005
```

La Session-Independent Workflow Execution viene collegata a:

```
DEC-012
+ PROT-009
+ PROC-005 / PROC-006 / PROC-007
```

Il capability routing usa:

```
PROT-011
+ PROT-003
```

Queste relazioni permettono al sistema di non trattare ogni protocollo come un elemento isolato.

Quando WCM identifica il processo, può scoprire i protocolli collegati.

Quando identifica un evento, può scoprire protocolli che attraversano il processo corrente. Il routing finale nasce dall'intersezione.

---

## 14.7 Knowledge nodes → Procedure relationships

Non tutto parte da un processo.

Anche un nodo di conoscenza può indicare che una procedura è necessaria.

Immaginiamo di aprire un documento e trovare:

```
STATUS = SUPERSEDED
```

Questo metadata non è soltanto informazione descrittiva.

Può influenzare il routing:

```
SUPERSEDED
→ non usare come baseline corrente
→ cerca il successore autorevole
```

Oppure un nodo può dichiarare:

```
DEPENDS_ON
AFFECTS
CONSTRAINS
EVIDENCE_FOR
```

Queste relazioni possono indicare:

- quale protocollo verifica l'integrità;
- quale processo deve essere richiamato;
- quale nodo è affected da un delta;
- quale evidence serve prima della promozione.

Per questo il Knowledge Navigation Layer e il Protocol Routing sono collegati.

```
KNOWLEDGE GRAPH
→ dice cosa è collegato a cosa

PROTOCOL ROUTING
→ dice quali regole diventano operative
```

Una relazione nella memoria non è automaticamente un comando.

Ma può essere una route verso la procedura applicabile.

---

## 14.8 Guard deterministici

Arriviamo al punto in cui WCM può ridurre il margine interpretativo.

La baseline corrente mantiene una sorgente machine-readable:

`wcm/runtime/protocol-routing/ROUTING_SOURCE.json`

Essa contiene route nella forma:

```
EVENT
+
HOOK
→ LOAD
→ ACTION
→ SERVICE POLICY
```

Esempio reale della baseline corrente:

```
EVENT = TOOL_OUTPUT_LIMIT
HOOK  = ON_TOOL_FAILURE

LOAD
PROT-011
PROT-003
PROT-009

ACTION
RESOLVE_CAPABILITY

SERVICE POLICY
SERVICE_OPTIONAL
```

Qui l'agente non deve inventare la route.

L'evento è noto.

L'hook è noto.

La combinazione identifica la route.

Un altro esempio:

```
EVENT = BOARD_GATE_READY
HOOK  = BEFORE_BOARD_GATE

LOAD
PROT-010
PROT-009

ACTION
WAIT_AUTHORITY
```

Oppure:

```
EVENT = MATERIAL_DELTA
HOOK  = AFTER_MATERIAL_DELTA

LOAD
PROC-006
PROC-011

ACTION
RECONCILE
```

Queste route sono **guard deterministiche** in un senso preciso:

**quando l'evento e l'hook strutturati coincidono con una route dichiarata, il set da caricare non viene ricostruito per similarità semantica.**

Viene letto.

---

## 14.9 Perché servono sia Source sia Registry

Il runtime mantiene anche:

`PROTOCOL_ROUTING_REGISTRY.json`.

La Source e il Registry hanno funzioni differenti.

### ROUTING\_SOURCE

È la dichiarazione corrente delle route.

Contiene:

- event;
- hook;
- target da caricare;
- action;
- service policy;
- note.

### PROTOCOL\_ROUTING\_REGISTRY

È una rappresentazione materializzata che aggiunge controllo sui target.

Per ogni ID instradabile registra:

- path;
- status;
- appartenenza alla baseline corrente.

Il registry permette quindi di verificare una proprietà importante:

**il protocollo che sto per caricare esiste davvero nel path canonico ed è ACTIVE?**

Il routing corretto non è:

```
EVENT
→ nome protocollo ricordato
→ esegui
```

È:

```
EVENT + HOOK
→ ROUTING SOURCE
→ target IDs
→ REGISTRY / path / status
→ documento canonico
→ regola applicabile
```

Questa catena riduce tre failure mode:

1. protocollo inventato;
2. path inventato;
3. protocollo stale o non corrente.



---

## 14.10 Exact event + exact hook

Per le route strutturate la baseline corrente vieta il fuzzy matching.

Questo punto merita attenzione.

Immaginiamo due eventi:

```
CAPABILITY_UNVERIFIED
TOOL_OUTPUT_LIMIT
```

Entrambi riguardano una difficoltà tecnica.

Ma non sono identici.

Il primo carica:

```
PROT-011 + PROT-003
```

Il secondo carica:

```
PROT-011 + PROT-003 + PROT-009
```

Perché nel secondo caso può essere necessario preservare anche la continuità del workflow durante una failure.

Un sistema che raggruppasse genericamente tutto sotto:

! *«problema con tool»*

potrebbe perdere questa differenza.

Per questo, nel routing deterministico:

```
SIMILE
≠
UGUALE

EVENT NAME MATCH
+
HOOK MATCH
=
ROUTE APPLICABILE
```

L'exact matching non significa che ogni evento del mondo debba essere già formalizzato.

Significa che **quando un evento formalizzato esiste, non deve essere sostituito da un'approssimazione cognitiva.**

---

## 14.11 Gli hook: quando applicare la regola

L'evento dice **che cosa è successo.**

L'hook dice **in quale punto del ciclo operativo la route deve essere valutata.**

Nella baseline corrente incontriamo hook come:

```
ON_WAKE
ON_TOOL_FAILURE
```

```
BEFORE_STOP
AFTER_MATERIAL_DELTA
BEFORE_BOARD_GATE
```

Consideriamo **MEMORY\_OR\_INDEX\_DRIFT**.

La route è dichiarata con:

```
HOOK = BEFORE_STOP
```

Questo significa che, prima di trasformare il drift in una conclusione definitiva, WCM deve caricare:

```
PROC-008
PROT-013
PROT-005
```

e tentare la reconciliation prevista.

L'hook impedisce quindi che la regola venga applicata nel momento sbagliato.

```
EVENTO CORRETTO
+
MOMENTO SBAGLIATO
≠
ROUTE CORRETTA
```

## 14.12 Un evento può richiedere processi e protocolli

Il nome **Protocol Routing** potrebbe far pensare che la route carichi soltanto protocolli.

Non è così.

**ROUTING\_SOURCE.json** può caricare anche processi.

Esempio:

```
MATERIAL_DELTA
→ PROC-006
→ PROC-011
```

oppure:

```
MEMORY_OR_INDEX_DRIFT
→ PROC-008
→ PROT-013
→ PROT-005
```

Questo riflette una proprietà importante del WCM:

**| un evento non appartiene necessariamente a una sola categoria documentale.**

Per affrontarlo correttamente possono servire:

- un processo che descrive il flusso;
- un protocollo che impone la guard;
- una decisione che stabilisce authority o precedence;
- uno stato runtime che dice dove siamo.

Il routing trova il bundle minimo applicabile.

---

## 14.13 Service policy

Le route strutturate possono anche dichiarare una `service_policy`.

Nella baseline corrente troviamo, per esempio:

```
NONE
SERVICE_OPTIONAL
```

Questo campo non sceglie automaticamente un service.

Definisce il perimetro.

### NONE

La route non prevede service per la normale risoluzione.

### SERVICE\_OPTIONAL

La route può arrivare a una delega, ma soltanto dopo i controlli precedenti.

Esempio:

```
TOOL_OUTPUT_LIMIT
→ verifica capability diretta
→ Direct Before Delegate
→ preserva continuità workflow
→ service solo se realmente necessario
```

Questo evita il salto:

```
problema tecnico
→ delega immediata
```

La policy di service fa quindi parte del routing, non è una decisione separata presa dopo.

---

## 14.14 Interpretazione cognitiva

Il routing deterministico funziona soltanto dopo che abbiamo un evento strutturato.

Ma chi riconosce l'evento?

Non sempre esiste una risposta meccanica.

Una richiesta umana come:

**|** *«La pagina non sembra aggiornata.»*

può significare:

- projection stale;
- cache;
- runtime non riconciliato;
- release non pubblicata;

- errore dell'utente;
- problema di rete.

Prima di associare un evento canonico, WCM deve comprendere il contesto.

Questa è una funzione cognitiva.

```
FRASE AMBIGUA
↓
REASONING
↓
EVIDENCE
↓
EVENTO STRUTTURATO, se giustificato
↓
ROUTE DETERMINISTICA
```

L'errore opposto sarebbe assegnare troppo presto un event ID soltanto perché alcune parole sembrano simili.

Il sistema diventerebbe formalmente deterministico ma semanticamente sbagliato.

## 14.15 Reasoning vs mechanical enforcement

Questo è uno dei confini più importanti dell'architettura WCM.

### Il reasoning serve quando:

- dobbiamo interpretare un'intenzione;
- dobbiamo classificare un caso nuovo;
- dobbiamo comprendere una contraddizione semantica;
- dobbiamo capire se un problema osservato corrisponde davvero a un evento canonico;
- dobbiamo valutare sufficienza del contesto;
- dobbiamo distinguere RUN da CHANGE in base al significato materiale.

### Il mechanical enforcement serve quando:

- il workflow status è già strutturato;
- l'event ID è già strutturato;
- l'hook è già noto;
- il registry espone path e status;
- uno schema deve essere validato;
- una projection deve essere derivata da input strutturati;
- una regola exact-match è già stata formalizzata.

La formula è:

```
USA REASONING
PER SCOPRIRE IL SIGNIFICATO

USA DETERMINISMO
PER NON REINTERPRETARE
```

Il WCM non elimina il reasoning.  
Lo circoscrive.

---

## 14.16 Cosa succede se la route non esiste?

Non ogni situazione possibile è già presente in `ROUTING_SOURCE.json`.

Se un evento non ha una route strutturata, WCM non deve inventare un ID per far sembrare il sistema completo.

Si torna al routing generale:

```
SITUAZIONE NON COPERTA
↓
INDEX-FIRST
↓
Process Register / Protocol Book / KB
↓
Source Precedence
↓
processi e protocolli realmente applicabili
```

Se emerge che la baseline possiede un gap metodologico, quel fatto può diventare evidence.

Ma non autorizza una modifica automatica del metodo.

```
ROUTE MANCANTE
≠
PERMESSO DI CREARE UNA NUOVA ROUTE
```

Aggiungere materialmente una nuova route al routing canonico sarebbe un WCM CHANGE e richiederebbe il relativo gate.

---

## 14.17 Drift tra Source e Registry

Esiste un caso delicato.

La Source dichiara una route.

Il Registry materializzato potrebbe essere stale.

Oppure un target potrebbe non esistere più.

La baseline corrente prevede che il sistema non debba scegliere a intuito.

Se Source e Registry divergono:

```
ROUTING_SOURCE
+
verifica target ACTIVE
→ riferimento operativo

REGISTRY STALE
→ MEMORY_OR_INDEX_DRIFT
```

→ reconciliation

L'obiettivo è evitare che una cache o una vista materializzata acquisisca authority superiore alla propria source.

È la stessa logica vista nel Capitolo 12:

SOURCE OF TRUTH  
→ materializzazione  
→ presentation  
  
NON IL CONTRARIO

---

## 14.18 Esempio 1 — Resume al wake

Evento strutturato:

WAKE\_RESUME\_REQUIRED

Hook:

ON\_WAKE

Route corrente:

LOAD  
PROC-005  
PROT-005  
PROT-009  
  
ACTION  
CONTINUE

Il significato è:

1. eseguire il bootstrap;
2. recuperare il minimo contesto autorevole;
3. applicare Resume Priority;
4. continuare dal checkpoint senza duplicare gli step già completati.

Il sistema non ha dovuto caricare l'intero Protocol Book.

Ha caricato il bundle necessario a quel wake.

---

## 14.19 Esempio 2 — Dipendenza interna ancora pending

Evento:

INTERNAL\_DEPENDENCY\_PENDING

Hook:

ON\_WAKE

Route:

```
PROT-018  
PROT-009
```

Action:

```
WAIT_INTERNAL_RESOLUTION
```

Il routing impedisce una falsa classificazione.

La dipendenza è nota come interna al WCM.

Quindi:

```
PENDING  
≠  
BLOCKER DI PROGETTO  
≠  
GATE UMANO  
≠  
AUTORIZZAZIONE A RIFARE IL LAVORO
```

Questa precisione deriva dal fatto che l'evento è stato formalizzato.

---

## 14.20 Esempio 3 — Stato operativo sconosciuto

Evento:

```
UNKNOWN_OPERATIONAL_STATE
```

Hook:

```
BEFORE_STOP
```

Route:

```
PROT-016  
PROC-011
```

Action:

```
FAIL_CLOSED
```

Il sistema non deve dedurre lo stato con regex o fuzzy parsing quando il contratto strutturato è incoerente.

Prima si tenta la reconciliation deterministica.

Se il fatto esecutivo resta ambiguo, il sistema non inventa uno stato plausibile.

---

## 14.21 Esempio 4 — Drift della conoscenza

Evento:

```
MEMORY_OR_INDEX_DRIFT
```

Hook:

```
BEFORE_STOP
```

Route:

```
PROC-008  
PROT-013  
PROT-005
```

Action:

```
RECONCILE
```

Questa route combina tre funzioni:

- assurance della conoscenza;
- health delle relazioni;
- retrieval autorevole.

È un buon esempio del fatto che il bundle procedurale non coincide necessariamente con un singolo protocollo.

---

## 14.22 Esempio 5 — Material delta

Evento:

```
MATERIAL_DELTA
```

Hook:

```
AFTER_MATERIAL_DELTA
```

Route:

```
PROC-006  
PROC-011
```

Action:

```
RECONCILE
```

Dopo un cambiamento materiale, il sistema deve chiedersi che cosa deve sopravvivere e quali viste devono essere riconciliate.

La route non dice:

**|** «*riscrivi tutto*».

Dice quali processi governano la continuità dopo il delta.

---

## 14.23 Esempio 6 — Board Gate pronto

Evento:

```
BOARD_GATE_READY
```



Hook:

```
BEFORE_BOARD_GATE
```

Route:

```
PROT-010  
PROT-009
```

Action:

```
WAIT_AUTHORITY
```

La route collega due esigenze:

- contract corretto dell'authority command;
- vera stop condition del workflow.

Il risultato non è un'azione autonoma.

È una **fermata governata**.

Questo mostra che il routing non serve soltanto a capire cosa fare.

Serve anche a capire **quando non fare il passo successivo**.

---

## 14.24 Un protocollo non deve apparire dal nulla

Quando leggiamo una risposta prodotta da un agente, può essere difficile capire da dove arrivi una regola.

Il WCM cerca di preservare la provenance anche nel routing.

Una route corretta dovrebbe essere ricostruibile:

```
RICHIESTA / EVENTO  
↓  
CLASSIFICAZIONE  
↓  
HOOK  
↓  
ROUTING SOURCE o INDEX  
↓  
PROCESS / PROTOCOL ID  
↓  
PATH CANONICO  
↓  
STATUS  
↓  
REGOLA APPLICATA
```

Questo rende possibile una domanda fondamentale:

■ **«Perché hai applicato proprio questo protocollo?»**

La risposta non dovrebbe essere:

■ *«Perché mi sembrava adatto.»*

Dovrebbe poter indicare la route.

---

## 14.25 Anti-pattern

### Anti-pattern 1 — Caricare tutti i protocolli

Più regole nel contesto non significa più sicurezza.

### Anti-pattern 2 — Protocollo scelto per similarità del nome

Un nome semanticamente vicino non equivale a una route valida.

### Anti-pattern 3 — Fuzzy event routing

Per una route strutturata, `TOOL_OUTPUT_LIMIT` non diventa genericamente «tool problem».

### Anti-pattern 4 — Registry come authority autonoma

Il registry è materializzazione della routing source, non una nuova fonte superiore.

### Anti-pattern 5 — Evento inventato

Se la situazione non è formalizzata, usare il routing generale. Non creare un event ID ad hoc.

### Anti-pattern 6 — Determinismo prematuro

Trasformare una frase ambigua in evento strutturato senza evidence sufficiente.

### Anti-pattern 7 — Reasoning dove esiste una primitive

Reinterpretare con LLM un exact event + exact hook già dichiarato.

### Anti-pattern 8 — Mechanical rule per un conflitto semantico

Tentare di risolvere automaticamente un conflitto di significato che richiede authority o reasoning.

---

## 14.26 La formula compatta

Il routing dei protocolli può essere riassunto così:

1. CHE TIPO DI OPERAZIONE / EVENTO HO?
2. ESISTE UN EVENTO STRUTTURATO?
3. QUAL È L'HOOK?
4. ESISTE UNA ROUTE EXACT-MATCH?
5. QUALI TARGET DICHIARA?
6. I TARGET ESISTONO ED SONO CURRENT / ACTIVE?
7. CHE ACTION / GUARD / SERVICE POLICY IMPONGONO?
8. SE NON ESISTE ROUTE, QUAL È IL PERCORSO INDEX-FIRST MINIMO?
9. HO RAGGIUNTO IL CONTESTO SUFFICIENTE?
10. ESISTE UNA TRUE STOP?

In una sola espressione:

```
COGNITIVE CLASSIFICATION
+
EXACT ROUTING WHEN AVAILABLE
+
ACTIVE TARGET VERIFICATION
+
MINIMUM AUTHORITATIVE RETRIEVAL
=
APPLICABLE PROCEDURAL CONTEXT
```

## 14.27 Cosa abbiamo ottenuto

Nei Capitoli 13 e 14 abbiamo trasformato una richiesta in un percorso governato.

```
REQUEST
↓
INTENT / GOAL / SCOPE
↓
AUTHORITY
↓
RUN / CHANGE
↓
PROCESS
↓
EVENT / HOOK
↓
PROTOCOL ROUTING
↓
GUARD / ACTION / STOP
↓
EXECUTION
```

Ora WCM non deve conoscere a memoria tutti i protocolli per usarli.

Deve saper **riconoscere la situazione**, **navigare la baseline** e, quando il significato è già stato strutturato, **smettere di reinterpretarlo**.

Il Capitolo 15 chiuderà questa parte con esempi completamente domain-agnostic.

Non descriveremo un singolo progetto reale.

Vedremo invece richieste astratte e seguiremo, una per una, le catene:

```
REQUEST
→ PROCESS
→ PROTOCOL
→ GUARD
→ ACTION
→ STOP
```

## Source Map

## Fonti canoniche principali

- [WCM\\_AGENT\\_START.md](#) — bootstrap, Resume Priority, capability routing, exact event routing e structured-before-text;
- [wcm/kb/concepts/CONCEPT-007\\_AGENT\\_READY\\_KNOWLEDGE\\_ARCHITECTURE.md](#) — Knowledge Navigation Layer, Progressive Retrieval e source precedence;
- [wcm/process-book/PROCESS\\_REGISTER.md](#) — baseline corrente di 12 processi / 20 protocolli e relazioni operative principali;
- [wcm/runtime/protocol-routing/ROUTING\\_SOURCE.json](#) — source corrente delle route event + hook → load + action + service\_policy;
- [wcm/runtime/protocol-routing/PROTOCOL\\_ROUTING\\_REGISTRY.json](#) — materializzazione dei target instradabili con path e status ACTIVE;
- [wcm/process-book/processes/PROC-005\\_AGENT\\_READY\\_CONTEXT\\_BOOTSTRAP.md](#) — bootstrap e Context Sufficiency;
- [wcm/process-book/protocols/PROT-005\\_INDEX\\_FIRST\\_PROGRESSIVE\\_RETRIEVAL.md](#) — retrieval progressivo;
- [wcm/process-book/protocols/PROT-009\\_CONTIGUOUS\\_WORKFLOW\\_EXECUTION.md](#) — Resume Priority, true stop e Completion Gate;
- [wcm/process-book/protocols/PROT-011\\_CAPABILITY\\_EVIDENCE\\_CHECK\\_BEFORE\\_BLOCK.md](#) — capability evidence;
- [wcm/process-book/protocols/PROT-016\\_DETERMINISTIC\\_STATE\\_PROJECTION.md](#) — structured-before-text per execution state.

## Relazioni

```
CH14
├ CONTINUES → CH13
├ DERIVED_FROM → CONCEPT-007
├ MAPS → PROCESS_REGISTER
├ EXPLAINS → ROUTING_SOURCE.json
├ VERIFIED_BY → PROTOCOL_ROUTING_REGISTRY.json
├ GOVERNED_BY → PROT-005
├ GOVERNED_BY → PROT-009
└ PREPARES → CH15
```

## Maturity note

Il Protocol Routing machine-readable è parte della baseline WCM implementata per gli eventi operativi formalizzati nel registry corrente. Questo non implica che ogni possibile situazione sia già codificata né che la classificazione semantica di una richiesta sia deterministica. L'exact routing riduce l'ambiguità **dopo** che l'evento è stato sufficientemente identificato; la field validation complessiva del modello continua.